

LAB # 6

Deadlock in concurrency:

OBJECTIVE:

Implementing multiple thread blocked resources with help of lock and deadlock conditions.

Lab Task:

Create three threads by implementing thread synchronization block through 3 locks. (Hint: Apply un-sequenced lock to analyze deadlock and solve it through provided solution:

TASK:

```
package SEM;
public class Lab6 {
    private static final Object lockA = new Object();
    private static final Object lockB = new Object();
    private static final Object lockC = new Object();
    public static void main(String[] args) {
        Runnable task = () -> {
            String name = Thread.currentThread().getName();
            synchronized (lockA) {
                System.out.println(name + " acquired Lock A");
                sleep(100);
                synchronized (lockB) {
                    System.out.println(name + " acquired Lock B");
                    sleep(100);
                    synchronized (lockC) {
                        System.out.println(name + " acquired Lock C");
                        sleep(100);
                        System.out.println(name + " released all Locks");
                    }
                }
            }
        };

        Thread t1 = new Thread(task, "Affan");
        Thread t2 = new Thread(task, "021");
        Thread t3 = new Thread(task, "Thread-3");

        t1.start();
        t2.start();
        t3.start();
    }

    private static void sleep(int ms) {
        try { Thread.sleep(ms); } catch (InterruptedException ignored) {}
    }
}
```

OUTPUT:

```
Affan acquired Lock A
Affan acquired Lock B
Affan acquired Lock C
Affan released all Locks
Thread-3 acquired Lock A
Thread-3 acquired Lock B
Thread-3 acquired Lock C
Thread-3 released all Locks
021 acquired Lock A
021 acquired Lock B
021 acquired Lock C
021 released all Locks
```